



ELISA
Enabling **Linux** in
Safety Applications

WORKSHOP

What approach can we recommend for Enabling Linux in Safety Applications?

Paul Albertella (*Codethink*)

Igor Stoppa (*NVIDIA*)

Pete Brink (*UL*)



Introduction

- Open Source Engineering Process (OSEP) working group
 - Can open source software projects apply the processes expected by safety standards?
 - Can these processes be applied to open source as part of safety application projects?
 - Can these safety projects benefit from using open source tools and processes?

Our Goal: Create reusable resources for use by safety application projects involving Linux and describe how these can help

Why are we here?

- We want to avoid software license costs and vendor lock-in
- Linux supports a new technology that we want to use in our product
- We need the performance it can achieve when using our hardware

But...

- What are the risks of using Linux in safety applications?
- No warranty (including implied warranty of fitness for a particular purpose)
- We need a safety certificate to include this software in our product
- Simply “supporting” a technology isn’t good enough

Common questions from newcomers to ELISA

- **Isn't Linux “proven in use”?** i.e. Are the well-established uses of Linux in other contexts and domains relevant when making claims about it in safety applications?
- **Doesn't Linux follow the standards?** i.e. Can safety projects make claims based on the processes followed by the Linux project and other open source communities?
- **Can't you tell them to follow the standards?** i.e. Can ELISA encourage the Linux kernel community to adopt more rigorous processes that might strengthen such claims?
- **Can't you make a safe Linux?** i.e. Can ELISA create a ‘special’ Linux or specific resources that projects can use to make claims about Linux in safety applications?

The short answer to all of these? No.

The long(er) answers

- The use of Linux in other domains can justify **why** we want to use Linux, but cannot be the basis of a '**proven in use**' argument.
- The Linux project has examples of **good practices** that can help, but these cannot be the basis of a **process-driven** argument.
- ELISA has attempted to promote **further good practices** in the Linux community, but with **limited success**.
- ELISA can document systemic **shortcomings** or **gaps**, then users can decide how these apply to their **specific use case** for Linux - and how to deal with them.

Why? Because **safety is system-specific**

Where does that leave us?

- Make a case for Linux as a **safety element *in context***
 - **Analyse** the roles that Linux *must* play in **your** system
 - **Minimise** its responsibilities in your safety application design
 - Understand how it can **interfere** with your application's safety goals
 - Include other elements to **mitigate** this interference

What approaches are being tried? (1)

- Qualified hypervisor with a monitor running in a separate trusted partition side-by-side with the kernel
 - Monitor needs to have visibility of key application, OS or system properties
 - Observation requirements depend on the nature of the safety functions
- Use a qualified set of components within the kernel
 - Needs proven freedom from interference to perform the monitoring function
 - This involves support from a hypervisor and hardware
 - Gives unconstrained access to everything in the kernel, but you need to provide the independence from the failure modes of the monitored function

What approaches are being tried? (2)

- User space monitor with an external hardware watchdog
 - Observing critical system properties to ensure that they are within bounds
 - Receiving data / signals from safety-critical tasks to confirm that they are operating correctly and performing within tolerances
 - Monitor is subject to the interference it is responsible for detecting (but can therefore operate as a 'canary' for some types)
 - Reliance on external mitigation can create availability problems

Key challenges

- How independent is your detection/mitigation?
 - Can it be affected by the interference that it is supposed to detect?
- The kernel is its own worst enemy
 - It can trample all over anything when it goes wrong - so prepare for the worst!
- What is “the system” for you and what does “safety” mean for it?
 - Wider system context *will* have a very radical effect on your software and system behaviour!
 - “Out of context” really means “for an assumed context”
- What does “the long term” mean for your system?
 - The CRA expects at least 5 years, automotive 15 years or more, rail even longer

So what can ELISA do?

- Document the **challenges** faced when integrating Linux as part of a system
- Document **limitations** in the architecture or design of Linux
- Identify other system **elements** that could be part of a given solution
- Identify **processes** that anyone using Linux in a safety context should follow
- Describe **approaches** that creators of Linux-based systems can use
- Provide **examples** of how these approaches can be applied in practice
- Provide example **use cases** and why different aspects may be relevant

Other ELISA working groups

- Does this perspective align with the work in other working groups?
- What resources can ELISA usefully create?

Resources: Open Projects

- What resources can we **build on** from other open projects?
 - e.g. LTP, stress-ng
- What can we **learn** from other open projects?
 - e.g. Xen, Zephyr

Questions / Discussion



Licensing of Workshop Results

All work created during the workshop is licensed under Creative Commons Attribution 4.0 International (CC-BY-4.0) [<https://creativecommons.org/licenses/by/4.0/>] by default, or under another suitable open-source license, e.g., GPL-2.0 for kernel code contributions.

You are free to:

- Share — copy and redistribute the material in any medium or format
- Adapt — remix, transform, and build upon the material for any purpose, even commercially.

The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms:

Attribution — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

No additional restrictions — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.